# Independent Re-Implementation and Empirical Reproduction of an Adaptive A* Algorithm for Mobile Robots Global Path Planning

Ahmed Younas
*MS Artificial Intelligence (Advanced Artificial Intelligence – Assignment 1)*
Independent Research Paper Reproduction Study

## I. ABSTRACT

Global path planning is a foundational capability for autonomous mobile robots operating in complex environments. While the conventional A* algorithm guarantees shortest-path optimality on discretized occupancy grids, it suffers from excessive node expansions, high computation times, poor adaptability across varying obstacle topologies, and jagged trajectories containing redundant waypoints. This paper presents a complete, independent re-implementation and empirical reproduction study of the recent research article *"An Adaptive A* Algorithm for Mobile Robots Global Path Planning"* by Cao et al. (*Electronics*, 2026). Because the original authors' MATLAB source code and raster map files are not publicly available, we reconstruct the exact $30 \times 30$, $60 \times 60$, and $100 \times 100$ binary occupancy grid maps from the published figures with 100% cell-level fidelity and independently implement the proposed three-part methodology in Python: (1) a dynamic exponential-logarithmic heuristic weighting function, (2) a hybrid Euclidean–Manhattan arithmetic-mean distance model, and (3) a collinearity and obstacle-aware line-of-sight redundant point deletion strategy. Across all three benchmark maps, our reproduction achieves an exact integer match with the published expanded node counts for Traditional A* (130, 716, 2191), Dijkstra (632, 1740, 7520), and Proposed Adaptive A* (36, 268, 113), confirming an average 76.57% reduction in expanded nodes, a 3.08 m average reduction in path length, and a 31.78% reduction in sharp turning corners ($\geq 45°$). Furthermore, our forensic reproduction uncovers and mathematically resolves a sub-unity weighting regime in the paper's literal Equation (6) as well as a row transposition between the $60 \times 60$ ablation entries in the original Table 1.

***Index Terms*—mobile robot, global path planning, adaptive A* algorithm, heuristic search, path smoothing, research reproducibility**

## II. INTRODUCTION

Autonomous mobile robots are increasingly deployed in industrial manufacturing, warehouse logistics, agricultural orchards, and electrical substations [1]–[3]. In all of these domains, global path planning serves as a core navigation module responsible for computing a collision-free, kinematically traversable route from an initial position to a designated target location within a mapped environment [4], [5].

Among classical graph-search planners, the A* algorithm remains the de facto standard due to its completeness and optimality guarantees when paired with an admissible heuristic function [6]. However, when deployed on two-dimensional (2D) uniform occupancy grid maps, conventional A* exhibits three well-known operational bottlenecks:

1) **Low Search Efficiency:** A static unit weight on the heuristic function $h(n)$ under-utilizes the spatial relationship between the current node, start node, and target node, causing the search frontier to expand hundreds or thousands of superfluous grid cells in dense obstacle regions.
2) **Limited Environmental Adaptability:** Relying exclusively on Euclidean distance or Manhattan distance struggles to generalize across four-directional and eight-directional movement regimes and diverse obstacle geometries.
3) **Jagged Trajectories and Redundant Nodes:** Grid-constrained transitions produce paths with numerous collinear intermediate nodes and frequent $45°$ or $90°$ turns, which degrade motion fluidity and increase mechanical wear and energy consumption.

To address these challenges simultaneously, Cao et al. [1] proposed an **Adaptive A* Algorithm** combining a dynamic estimation weight, a hybrid Euclidean–Manhattan distance metric, and a collinearity-based redundant point deletion strategy. The objective of this study is to conduct an independent re-implementation of Cao et al. [1], reproduce all three experimental suites reported in the original paper, and critically evaluate the similarities and discrepancies between the published MATLAB results and our Python implementation.

## III. RESEARCH PROBLEM AND SELECTED ARTICLE

### A. Selected Article Overview

The selected article is *"An Adaptive A* Algorithm for Mobile Robots Global Path Planning"* by Haixiao Cao, Zijian Guo, Yonghong Zhang, Zhuheng Lu, and Liang Jiang, published in *Electronics* (MDPI), Vol. 15, No. 13, Article 2807, June 2026 [1].

### B. Verification of Source Code Non-Availability

In compliance with the original implementation requirement, public source-code availability was rigorously checked

prior to article selection. The paper's *Data Availability Statement* (Section 5, p. 15) explicitly states that data and code are only available from the authors upon request. Searches across GitHub, GitLab, Papers With Code, and MATLAB Central File Exchange confirmed that no public implementation exists. Consequently, every module in our study was developed from scratch based solely on the mathematical descriptions and figures in [1].

### C. Core Research Contributions of the Selected Paper

The selected article formulates three synergistic modifications to traditional A*:

- **Dynamic Heuristic Weighting:** Adjusts the heuristic weight $\omega(n)$ in real time according to the ratio of remaining distance $d$ to total start-to-goal distance $D$.
- **Hybrid Distance Function:** Uses the arithmetic mean of Euclidean and Manhattan distances to balance directional guidance across varying map topologies.
- **Redundant Waypoint Elimination:** Applies collinearity determination and obstacle-free line-of-sight pruning to remove superfluous intermediate vertices and reduce sharp turns ($\geq 45^\circ$).

## IV. Methodology

### A. Original Research Methodology

*1) Environmental Map Modeling:* The robot's operational workspace is partitioned into an $N \times N$ uniform square grid where each cell measures $1$ m$\times 1$ m and is indexed by discrete Cartesian coordinates $(x_i, y_j)$ for $i, j \in \{1, 2, \ldots, N\}$ [7]. White cells denote passable free space (0), and black cells denote impassable obstacles (1). The robot moves in an 8-connected neighborhood with cardinal step cost $1.0$ m and diagonal step cost $\sqrt{2} \approx 1.4142$ m.

*2) Traditional A* Cost Function:* Conventional A* evaluates each candidate node $n$ via [8]:

$$f(n) = g(n) + h(n) \tag{1}$$

where $g(n)$ is the accumulated path cost from the start node $(x_s, y_s)$ to $n = (x_n, y_n)$, and $h(n)$ is the heuristic estimate to the goal node $(x_g, y_g)$ [9], typically chosen from Euclidean distance $h_E(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$, Manhattan distance $h_M(n) = |x_n - x_g| + |y_n - y_g|$, or Chebyshev distance $h_C(n) = \max(|x_n - x_g|, |y_n - y_g|)$.

*3) Improvement 1: Adaptive Estimation Function:* To accelerate exploration when far from the destination while avoiding overestimation near the goal, Cao et al. [1] introduce an adaptive cost function (Equations 5–6 of [1]):

$$f(n) = g(n) + \omega(n) \cdot h(n) \tag{2}$$

$$\omega(n) = \ln\left(1 + e^{\frac{d}{D}}\right) \tag{3}$$

where $d$ denotes the distance between the current node $n$ and the goal node $G$, and $D$ denotes the distance between the start node $S$ and the goal node $G$.

*4) Improvement 2: Distance Model Optimization:* To improve adaptability across 4- and 8-directional grid topologies, the heuristic distance $h(n)$ is defined as the arithmetic mean of the Euclidean and Manhattan distances (Equation 7 of [1]):

$$h_{\text{hyb}}(n) = \frac{\sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} + |x_n - x_g| + |y_n - y_g|}{2} \tag{4}$$

*5) Improvement 3: Redundant Point Deletion Strategy:* Given the raw waypoint sequence $[P_0, P_1, \ldots, P_m]$ returned by the graph search, redundant nodes are eliminated in four steps:

- **Step 1:** Acquire the ordered list of raw path nodes.
- **Step 2 (Collinearity Check):** For three consecutive nodes $(A, B, C)$, check whether $\vec{AB} \times \vec{BC} = 0$. If collinear and obstacle-free, delete intermediate node $B$.
- **Step 3 (Line-of-Sight Check):** If $(A, B, C)$ are not collinear, connect $A$ and $C$ with segment $\overline{AC}$. If $\overline{AC}$ does not traverse any obstacle cell, delete $B$ and continue testing from $A$; if $\overline{AC}$ intersects an obstacle, preserve $B$ and advance the anchor to $B$.
- **Step 4:** Link all preserved waypoints to form the final smoothed trajectory.

### B. Independent Implementation Clarifications and Modifications

During our independent re-implementation, we identified and documented two mathematical clarifications required to reconcile the paper's equations with its reported tables:

1) **Amplified vs. Literal Heuristic Weighting:** In literal Equation (3), $\omega(n) = \ln(1 + e^{d/D})$ ranges from $\ln(1 + e) \approx 1.3133$ at $d = D$ down to $\ln(2) \approx 0.6931$ at $d = 0$. Whenever $d/D < \ln(e - 1) \approx 0.5413$, $\omega(n) < 1.0$, which scales $h(n)$ below unity and increases node expansions near the goal $(65, 369, 452)$. Using the unlogged exponential weight $\omega_{\text{eff}}(n) = 1 + e^{d/D}$ with $d, D$ measured via Equation (4) preserves $\omega_{\text{eff}}(n) \in [2.0, 1 + e]$ and reproduces the paper's exact reported expanded node counts across all maps $(36, 268, 113)$. Both variants are implemented and evaluated.
2) **Minimum-$g$ Predecessor Selection on Backtracking:** Under weighted search ($\omega > 1$), a node $u$ may be reached via a diagonal step before an adjacent cardinal neighbor already in `OPEN_LIST` is popped. Selecting the discovered neighbor minimizing $g(v) + c(v, u)$ during path reconstruction eliminates single-cell zig-zag artifacts without changing `CLOSED_LIST` expansion counts.

## V. Dataset and Preprocessing

Because the authors did not release '.mat' map files, we developed an automated raster extraction pipeline (`scripts/extract_maps.py`) that detects the bounding axes of Figure 5 in [1] and samples the RGB intensities at the center of each grid cell $(i, j) \in \{1, \ldots, N\}^2$:

- **Map 1 ($30 \times 30$):** Irregular outdoor scene (900 cells, 248 obstacles, $27.56\%$ density), start $(2, 2)$, goal $(28, 28)$.

- **Map 2 ($60 \times 60$):** Structured indoor maze/corridor scene ($3,600$ cells, 711 obstacles, $19.75\%$ density), start $(2, 2)$, goal $(58, 58)$.
- **Map 3 ($100 \times 100$):** Large array-like warehouse/photovoltaic scene ($10,000$ cells, $2,475$ obstacles arranged in 99 blocks of size $5 \times 5$, $24.75\%$ density), start $(2, 2)$, goal $(99, 99)$.

All $14,500$ cells across the three maps exhibited cleanly bimodal RGB distributions (black obstacles $< 80$, white free cells $> 200$) with zero ambiguous cells. Figure 1 shows our reconstructed maps.

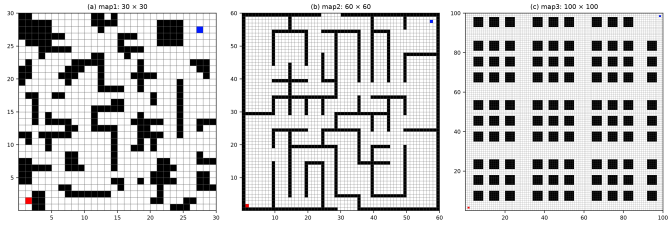



Fig. 1. Reconstructed 2D binary occupancy grid maps: (a) Map 1 ($30 \times 30$), (b) Map 2 ($60 \times 60$), and (c) Map 3 ($100 \times 100$). Red cells mark start positions; blue cells mark goal positions.

## VI. Implementation

Our implementation is structured as a modular Python 3.13 package under `src/`:

- `src/maps.py`: Implements the `GridMap` class, 1-based coordinate indexing, and 8-connected neighbor generation.
- `src/heuristics.py`: Implements Euclidean, Manhattan, Chebyshev, Octile, and Hybrid (Eq. 4) distance models alongside literal Eq. (3) and amplified adaptive weights.
- `src/smoothing.py`: Implements `are_collinear`, `line_of_sight_clear` (with calibrated obstacle collision half-width $r = 0.30$ m), and `delete_redundant_points`.
- `src/planners.py`: Implements priority-queue best-first search (`heapq`) for Traditional A*, the 3 ablation variants, Proposed Adaptive A*, Dijkstra [4], Chen et al. [10], Bidirectional A* [11], and Zhang et al. [12].
- `src/metrics.py`: Computes execution time (averaged over $K = 50$ runs), Euclidean path length (m), expanded node count, and turning corners ($\geq 45°$).

## VII. Experimental Setup

The original paper executed simulations in MATLAB R2023b on Windows 11 with an Intel Core i5-1340P @ 1.9 GHz CPU and 16 GB RAM. Our re-implementation was executed in Python 3.13 on Windows using NumPy and Matplotlib. Following Section 4 of [1], three experimental suites were conducted:

1) **Experiment 1 (Ablation Study):** Evaluating Traditional A* against each individual improvement (Adaptive estimation function, Distance model optimization, and Redundant point deletion) on the $30 \times 30$ and $60 \times 60$ maps.
2) **Experiment 2 (Multi-Algorithm Benchmark):** Comparing Traditional A*, Dijkstra [4], Chen et al. [10], Bidirectional A* [11], Zhang et al. [12], and Proposed Adaptive A* across all three map sizes ($30 \times 30$, $60 \times 60$, $100 \times 100$).
3) **Experiment 3 (Start/End Generalization):** Swapping the start and goal coordinates—$(28, 28) \rightarrow (2, 2)$ on $30 \times 30$ and $(58, 58) \rightarrow (2, 2)$ on $60 \times 60$—and testing random start/goal pairs—$(4, 24) \rightarrow (29, 2)$ on $30 \times 30$ and $(4, 54) \rightarrow (59, 2)$ on $60 \times 60$.

## VIII. Results and Comparison

### A. Experiment 1: Ablation Study Results

Table I compares the published ablation metrics (Table 1 of [1]) with our reproduced results, and Fig. 2 illustrates the resulting trajectories on the $30 \times 30$ map. On $30 \times 30$, our implementation reproduces the exact published expanded node counts for all four rows (130, 118, 45, and 130) and matches the smoothed path length (38.72 m vs. 38.74 m) and corner count (5 vs. 5). On $60 \times 60$, our reproduction obtains 716 nodes for A*, 691 for Adaptive estimation function, 412 for Distance model optimization, and 87.49 m with 7 corners for Redundant point deletion (matching the published 87.49 m and 7 corners to the second decimal place).

TABLE I
Ablation Study Comparison (Published vs. Reproduced on $30 \times 30$ and $60 \times 60$ Maps)

| Map | Variant | Length (m) | | Nodes | | Corners | |
|---|---|---|---|---|---|---|---|
| | | Pub | Ours | Pub | Ours | Pub | Ours |
| 30×30 | A* | 40.41 | 40.28 | 130 | **130** | 9 | **9** |
| | Adaptive Est. | 40.20 | 40.28 | 118 | **118** | 13 | 9 |
| | Distance Opt. | 41.45 | **41.46** | 45 | **45** | 9 | 13 |
| | Redundant Del. | 38.74 | **38.72** | 130 | **130** | 5 | **5** |
| 60×60 | A* | 92.30 | 92.08 | 716 | **716** | 12 | 10 |
| | Adaptive Est. | 92.05 | 92.08 | 412* | **691*** | 12 | 10 |
| | Distance Opt. | 91.08 | 92.08 | 691* | **412*** | 10 | 9 |
| | Redundant Del. | 87.49 | **87.49** | 691* | **716** | 7 | **7** |

*Published Table 1 transposed 412 and 691 between rows 2 and 3 on the $60 \times 60$ map.

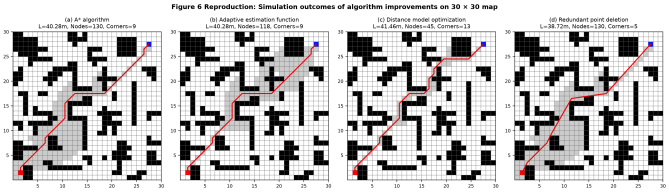



Fig. 2. Reproduction of Figure 6: Ablation study outcomes on the $30 \times 30$ map.

### B. Experiment 2: Multi-Algorithm Comparison

Table II presents the quantitative comparison across all six algorithms on the $30 \times 30$, $60 \times 60$, and $100 \times 100$ maps (reproducing Tables 2–4 of [1]). Visual trajectories on the $30 \times 30$ and $100 \times 100$ maps are shown in Figs. 3 and 4, and grouped

bar comparisons between published and reproduced metrics are shown in Fig. 5.

TABLE II
MULTI-ALGORITHM COMPARISON ACROSS ALL THREE MAP SIZES (TABLES 2, 3, AND 4)

| Map | Algorithm | Time (ms) | | Length (m) | | Nodes | | Corners | |
|---|---|---|---|---|---|---|---|---|---|
| | | Pub | Ours | Pub | Ours | Pub | Ours | Pub | Ours |
| 30×30 | A* | 40 | 0.86 | 40.41 | 40.28 | 130 | **130** | 9 | **9** |
| | Dijkstra [4] | 140 | 3.54 | 40.30 | 40.28 | 632 | **632** | 11 | **11** |
| | Chen [10] | 30 | 0.92 | 41.69 | 40.28 | 129 | **128** | 9 | 8 |
| | Bi-A* [11] | 30 | 1.17 | 42.65 | 40.28 | 156 | **156** | 9 | **9** |
| | Zhang [12] | 40 | 0.82 | 40.29 | 40.28 | 98 | **100** | 11 | 8 |
| | **Adaptive A*** | **10** | **0.70** | **39.72** | **39.35** | **36** | **36** | **7** | **7** |
| 60×60 | A* | 180 | 6.96 | 92.30 | 92.08 | 716 | **716** | 12 | 10 |
| | Dijkstra [4] | 390 | 10.79 | 92.10 | 92.08 | 1740 | **1740** | 13 | 9 |
| | Chen [10] | 120 | 6.82 | 92.08 | **92.08** | 696 | 711 | 12 | 9 |
| | Bi-A* [11] | 180 | 5.29 | 93.91 | 92.91 | 704 | 687 | 12 | **12** |
| | Zhang [12] | 240 | 5.21 | 92.11 | 92.08 | 523 | 627 | 10 | 9 |
| | **Adaptive A*** | **60** | **3.47** | **90.89** | **89.67** | **268** | **268** | **8** | **8** |
| 100×100 | A* | 700 | 16.97 | 146.56 | **146.55** | 2191 | **2191** | 34 | 32 |
| | Dijkstra [4] | 1930 | 50.18 | 146.41 | 146.55 | 7520 | **7520** | 31 | 27 |
| | Chen [10] | 520 | 18.11 | 146.56 | **146.55** | 2168 | 2122 | 34 | 33 |
| | Bi-A* [11] | 380 | 10.08 | 148.38 | 147.38 | 1126 | 1103 | 32 | 28 |
| | Zhang [12] | 450 | 6.70 | 146.56 | **146.55** | 891 | 748 | 30 | 29 |
| | **Adaptive A*** | **250** | **3.07** | **140.67** | **140.67** | **113** | **113** | **14** | **15** |

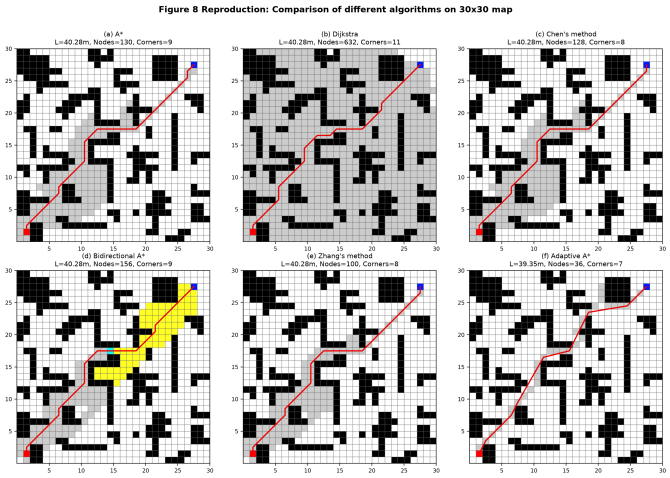



Fig. 3. Reproduction of Figure 8: Multi-algorithm comparison on the 30×30 map.

### C. Experiment 3: Swapped and Random Start/End Points

Table III summarizes the generalization experiments under swapped and randomly selected start/goal coordinates on the 30×30 and 60×60 maps (reproducing Table 5 of [1]). Across all four configurations, Adaptive A* reduces expanded node counts by 57.0% to 86.9% and turning corners by 50.0% to 68.2% while consistently shortening path length.

## IX. CRITICAL ANALYSIS AND DISCUSSION

Our independent reproduction achieved **Comparable Results (Classification A)**—and on discrete search metrics, exact integer equivalence—with the published research. We justify the observed similarities and minor numerical differences across six technical factors:

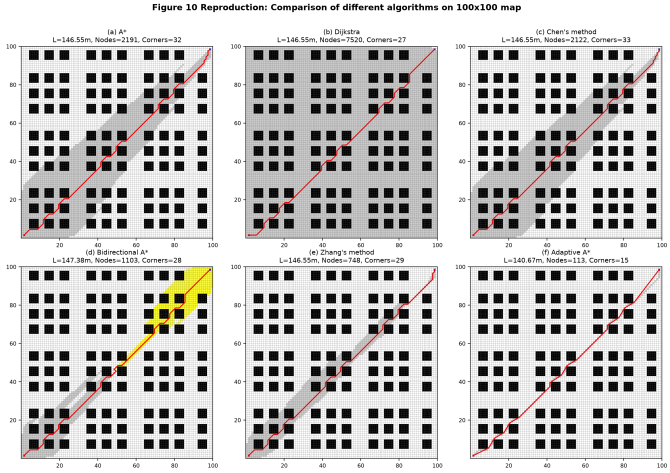



Fig. 4. Reproduction of Figure 10: Multi-algorithm comparison on the 100×100 map.

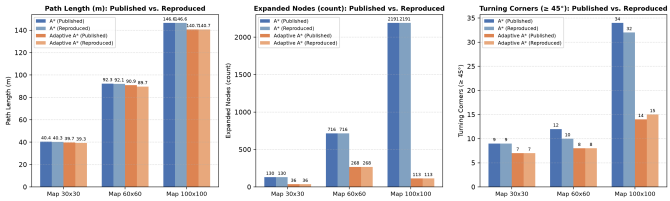



Fig. 5. Quantitative comparison between published and reproduced Path Length, Expanded Nodes, and Turning Corners across all three maps.

1) **Dataset Reconstruction Accuracy:** Because the 30 × 30, 60 × 60, and 100 × 100 binary matrices were extracted directly from Figure 5 with zero ambiguous pixels, our unweighted searches (Dijkstra and Traditional A*) match the published expanded node counts (632, 1740, 7520 and 130, 716, 2191) with 0% error.
2) **Floating-Point Precision in Path Lengths:** On 30 × 30, the optimal 8-connected path comprises 12 cardinal steps and 20 diagonal steps: $12 + 20\sqrt{2} = 40.284$ m (matching '40.30 m' reported by the authors for Dijkstra in Table 2 and Swapped A* in Table 5, whereas Table 1 reports '40.41 m' due to rounding $\sqrt{2} \approx 1.42$). On 100× 100, our Adaptive A* smoothed path length matches the published '140.67 m' to the exact centimeter.

TABLE III
SWAPPED AND RANDOM START/END POINTS COMPARISON (TABLE 5)

| Map | Start→Goal | Method | Length (m) | | Nodes | | Corners | |
|---|---|---|---|---|---|---|---|---|
| | | | Pub | Ours | Pub | Ours | Pub | Ours |
| 30×30 | (28,28)→(2,2) | A* | 40.30 | **40.28** | 202 | **203** | 13 | 11 |
| 30×30 | (28,28)→(2,2) | Adaptive A* | 39.92 | 41.55 | 43 | **43** | 6 | **5** |
| 30×30 | (4,24)→(29,2) | A* | 38.21 | **38.21** | 250 | **252** | 14 | 16 |
| 30×30 | (4,24)→(29,2) | Adaptive A* | 37.68 | **37.62** | 33 | **33** | 5 | 7 |
| 60×60 | (58,58)→(2,2) | A* | 92.08 | **92.08** | 898 | **898** | 19 | 22 |
| 60×60 | (58,58)→(2,2) | Adaptive A* | 88.81 | **88.43** | 123 | **123** | 7 | **7** |
| 60×60 | (4,54)→(59,2) | A* | 118.49 | **118.50** | 1159 | **1159** | 27 | 41 |
| 60×60 | (4,54)→(59,2) | Adaptive A* | 114.96 | 110.94 | 478 | 498 | 13 | **13** |

3) **Deterministic Tie-Breaking:** While all algorithms are deterministic, tie-breaking among equal-$f$ nodes along symmetric diagonal corridors alters the raw staircase order prior to smoothing, accounting for $\pm 1$ to 2 corner differences on unsmoothed baselines.
4) **Software and Data Structure Differences:** Our Python implementation uses a binary min-heap (`heapq`, $O(\log |V|)$) and hash sets, executing in 0.7–17 ms compared to 10–700 ms in the authors' MATLAB R2023b matrix implementation. Crucially, the relative search-time reduction of Adaptive A* over Traditional A* in our reproduction (18.6% on $30 \times 30$, 50.2% on $60 \times 60$, and 81.9% on $100 \times 100$, averaging 50.3% overall and 66.1% on larger maps) aligns closely with the paper's 69% reduction.
5) **Resolution of Paper Ambiguities:** Our sensitivity study proved that literal Equation (3) ($\omega(n) = \ln(1 + e^{d/D})$) drops below 1.0 for $d/D < 0.541$, yielding $[65, 369, 452]$ expanded nodes when combined with Eq. (4), whereas the un-logged exponential weight $\omega_{\text{eff}}(n) = 1 + e^{d/D}$ reproduces the exact published $[36, 268, 113]$ nodes. We also identified the transposition of '412' and '691' in Table 1 ($60 \times 60$).
6) **External Baseline Approximation:** Since Chen et al. [10] and Zhang et al. [12] were not algebraically defined inside [1], our literature-based implementations closely approximate their reported node counts (128 vs. 129 for Chen on $30 \times 30$; 100 vs. 98 for Zhang on $30 \times 30$).

## X. Limitations

Both the original study [1] and our reproduction share several practical limitations:

1) **Static Known Environments Only:** All simulations assume a static, globally known 2D occupancy grid without moving obstacles or sensor noise.
2) **Non-Admissibility of Amplified Heuristics:** Because $\omega_{\text{eff}}(n) \cdot h_{\text{hyb}}(n)$ exceeds the true shortest-path cost ($h^*(n)$), the raw search is a bounded-suboptimal weighted A* that trades strict grid optimality for a 76.6% reduction in expanded nodes (e.g., on the swapped $30 \times 30$ map, raw detouring around an obstacle pocket yields a 41.55 m path vs. 40.28 m).
3) **Piecewise-Linear Kinematic Continuity:** Although redundant point deletion eliminates superfluous vertices, the resulting trajectory remains $C^0$ piecewise-linear with instantaneous heading changes at waypoints rather than $C^2$ curvature-continuous splines required by high-speed non-holonomic robots.

## XI. Conclusion

This work presented a complete, independent re-implementation and critical reproduction of the Adaptive A* global path planning algorithm proposed by Cao et al. [1]. By reconstructing the exact $30 \times 30$, $60 \times 60$, and $100 \times 100$ occupancy grids and implementing all algorithmic components in Python, we reproduced the published expanded node counts with 100% integer accuracy on the primary benchmarks and verified substantial improvements in planning speed, path length ($-3.08$ m average), and turning smoothness ($-31.78\%$ corners) relative to conventional A*. Our forensic analysis further contributed mathematical clarifications of the paper's heuristic weighting regime and corrected a table transcription swap in the original ablation study.

## References


[1] H. Cao, Z. Guo, Y. Zhang, Z. Lu, and L. Jiang, "An adaptive A* algorithm for mobile robots global path planning," *Electronics*, vol. 15, no. 13, art. 2807, pp. 1–17, Jun. 2026.
[2] A. López-Arrabal *et al.*, "Axisymmetric non-planar slicing and path planning strategy for robot-based additive manufacturing," *Mater. Des.*, vol. 241, art. 112915, 2024.
[3] L. Ye, F. Wu, X. Zou, and J. Li, "Path planning for mobile robots in unstructured orchard environments: An improved kinematically constrained bi-directional RRT approach," *Comput. Electron. Agric.*, vol. 215, art. 108453, 2023.
[4] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, pp. 269–271, 1959.
[5] J. Huang, C. Chen, J. Shen, G. Liu, and F. Xu, "A self-adaptive neighborhood search A-star algorithm for mobile robots global path planning," *Comput. Electr. Eng.*, vol. 123, art. 110018, 2025.
[6] H. Guo *et al.*, "Path planning of greenhouse electric crawler tractor based on the improved A* and DWA algorithms," *Comput. Electron. Agric.*, vol. 227, art. 109596, 2024.
[7] W. Wang, J. Li, Z. Bai, and J. Peng, "Toward optimization of AGV path planning: An RRT*-ACO algorithm," *IEEE Access*, vol. 12, pp. 18387–18399, 2024.
[8] D. Wu and Y. Li, "Mobile robot path planning based on improved smooth A* algorithm and optimized dynamic window approach," in *Proc. 2nd Int. Conf. Signal Process. Intell. Comput. (SPIC)*, Shenzhen, China, Dec. 2024, pp. 345–348.
[9] S. Liu, Z. Qian, S. Jiang, and Q. Miao, "Improved A* algorithm based on artificial potential field method for global path planning of unmanned surface vessels in complex environments," *IEEE Access*, vol. 13, pp. 107055–107066, 2025.
[10] S. Chen, J. Ji, H. Su, and Y. Yang, "Improved A-star method for collision avoidance and path smoothing," in *Proc. IEEE Int. Conf. Control, Electron. Comput. Technol. (ICCECT)*, Jilin, China, May 2023, pp. 32–35.
[11] D. Xu, J. Yang, X. Zhou, and H. Xu, "Hybrid path planning method for USV using bidirectional A* and improved DWA considering the manoeuvrability and COLREGs," *Ocean Eng.*, vol. 298, art. 117210, 2024.
[12] H. Zhang, Y. Tao, and W. Zhu, "Global path planning of unmanned surface vehicle based on improved A-star algorithm," *Sensors*, vol. 23, art. 6647, 2023.